

ROCm environment variables

Print to PDF
Contents

- HIP environment variables
- ROCR-Runtime environment variables
- HIPCC environment variables
- Environment variables in ROCm libraries
- Key single-variable details

ROCm provides a set of environment variables that allow users to configure and optimize their development and runtime experience. These variables define key settings such as installation paths, platform selection, and runtime behavior for applications running on AMD accelerators and GPUs.

This page outlines commonly used environment variables across different components of the ROCm software stack, including HIP and ROCR-Runtime. Understanding these variables can help streamline software development and execution in ROCm-based environments.

HIP environment variables

The following tables list the HIP environment variables.

GPU isolation variables

 [latest](#) ▼

Restricting the access of applications to a subset of GPUs, also known as GPU isolation,

allows users to hide GPU resources from programs. The GPU isolation environment variables in HIP are collected in the following table.

Environment variable	Links	Value
ROCR_VISIBLE_DEVICES A list of device indices or UUIDs that will be exposed to applications.	GPU isolation, Setting the number of compute units	Example: 0, GPU-4b2c1a9f-8d3e-6f7a-b5c9-2e4d8a1f6c3b
GPU_DEVICE_ORDINAL Devices indices exposed to OpenCL and HIP applications.	GPU isolation	Example: 0, 2
HIP_VISIBLE_DEVICES Device indices exposed to HIP applications.	GPU isolation, HIP debugging	Example: 0, 2



Recommendation

- On Linux, use

ROCR_VISIBLE_DEVICES

.
- On Windows, use

HIP_VISIBLE_DEVICES

.

Profiling variables

The profiling environment variables in HIP are collected in the following table. For more information, check setting the number of CUs page.

Environment variable	Value
HSA_CU_MASK Sets the mask on a lower level of queue creation in the driver, this mask will also be set for queues being profiled.	Example: 1:0-8
ROC_GLOBAL_CU_MASK Sets the mask on queues created by the HIP or the OpenCL runtimes, this mask will also be set for queues being profiled.	Example: 0xf , enables only 4 CUs
HIP_FORCE_QUEUE_PROFILING Used to run the app as if it were run in rocprof. Forces command queue profiling on by default.	0: Disable 1: Enable

Debug variables

The debugging environment variables in HIP are collected in the following table. For more information, check [Logging HIP activity](#), [Debugging with HIP](#) and GPU isolation.

Environment variable	Default value	Value
AMD_LOG_LEVEL Enables HIP log on various level.	0	0: Disable log. 1: Enables error logs. 2: Enables warning logs next to lower-level logs. 3: Enables information logs next to lower-level logs. 4: Enables debug logs next to lower-level logs. 5: Enables debug extra logs next to lower-level logs.
AMD_LOG_LEVEL_FILE Sets output file for AMD_LOG_LEVEL .	stderr output	
AMD_LOG_MASK Specifies HIP log filters. Here is the complete list of log masks.	0x7FFFFFFF	0x1: Log API calls. 0x2: Kernel and copy commands and barriers. 0x4: Synchronization and waiting for commands to finish. 0x8: Decode and display AQL packets. 0x10: Queue commands and queue contents. 0x20: Signal creation, allocation, pool. 0x40: Locks and thread-safety code. 0x80: Kernel creations and arguments, etc. 0x100: Copy debug. 0x200: Detailed copy debug. 0x400: Resource allocation, performance-impacting events. 0x800: Initialization ar 0x1000: Misc debug, not yet classified. 0x2000: Show raw bytes of AQL packet.

Environment variable	Default value	Value
		0x4000: Show code creation debug. 0x8000: More detailed command info, including barrier commands. 0x10000: Log message location. 0x20000: Memory allocation. 0x40000: Memory pool allocation, including memory in graphs. 0x80000: Timestamp details. 0x100000: Comgr path information print. 0xFFFFFFFF: Log always even mask flag is zero.
HIP_FORCE_DEV_KERNARG Forces kernel arguments to be stored in device memory to reduce latency. Can improve performance by 2-3 μs for some kernels.	1	0: Disable 1: Enable
HIP_LAUNCH_BLOCKING Used for serialization on kernel execution.	0	0: Disable. Kernel executes normally. 1: Enable. Serializes kernel enqueue, behaves the same as AMD_SERIALIZE_KERNEL.
HIP_VISIBLE_DEVICES Only devices whose index is present in the sequence are visible to HIP	Unset by default.	0,1,2: Depending on the number of devices on the system.
GPU_DUMP_CODE_OBJECT Dump code object.	0	0: Disable 1: Enable
AMD_SERIALIZE_KERNEL Serialize kernel enqueue.	0	0: Disable 1: Wait for completion 2: Wait for completion after enqueue.

Environment variable	Default value	Value
		3: Both
AMD_SERIALIZE_COPY Serialize copies	0	0: Disable 1: Wait for completion before enqueue. 2: Wait for completion after enqueue. 3: Both
GPU_MAX_HW_QUEUES The maximum number of hardware queues allocated per device.	4	The recommended maximum value for this setting is 4. The variable controls how many independent hardware queues HIP runtime can create per process, per device. If an application allocates more HIP streams than this number, then HIP runtime reuses the same hardware queues for the new streams in a round-robin manner. Note that this maximum number does not apply to hardware queues that are created for CU-masked HIP streams, or cooperative queues for HIP Cooperative Groups (single queue per device).

Memory management related variables

The memory management related environment variables in HIP are collected in the following table. The `HIP_HOST_COHERENT` variable linked at the following pages:

- [Coherence control](#)
- [Memory allocation flags](#)

Environment variable	Default value	Value
HIP_HIDDEN_FREE_MEM Amount of memory to hide from the free memory reported by hipMemGetInfo.	0	0: Disable Unit: megabyte (MB)
HIP_HOST_COHERENT Specifies if the memory is coherent between the host and GPU in <code>hipHostMalloc</code> .	0	0: Memory is not coherent. 1: Memory is coherent. Environment variable has effect, if the following conditions are statisfied: - One of the <code>hipHostMallocDefault</code> , <code>hipHostMallocPortable</code> , <code>hipHostMallocWriteCombined</code> or <code>hipHostMallocNumaUser</code> flag set to 1. - <code>hipHostMallocCoherent</code> , <code>hipHostMallocNonCoherent</code> and <code>hipHostMallocMapped</code> flags set to 0.
HIP_INITIAL_DM_SIZE Set initial heap size for device malloc.	8388608	Unit: Byte The default value corresponds to 8 MB.
HIP_MEM_POOL_SUPPORT Enables memory pool support in HIP.	0	0: Disable 1: Enable
HIP_MEM_POOL_USE_VM Enables memory pool support in HIP.	0: other OS 1: Windows	0: Disable 1: Enable
HIP_VMEM_MANAGE_SUPPORT Virtual Memory Management Support.	1	0: Disable 1: Enable

Environment variable	Default value	Value
GPU_SINGLE_ALLOC_PERCENT Limits the maximum size of a single memory allocation as a percentage of GPU memory.	100	Unit: Percentage Prevents single allocations from consuming all available GPU memory.
GPU_MAX_HEAP_SIZE Set maximum size of the GPU heap to % of board memory.	100	Unit: Percentage
GPU_MAX_REMOTE_MEM_SIZE Maximum size that allows device memory substitution with system.	2	Unit: kilobyte (KB)
GPU_NUM_MEM_DEPENDENCY Number of memory objects for dependency tracking.	256	
GPU_STREAMOPS_CP_WAIT Force the stream memory operation to wait on CP.	0	0: Disable 1: Enable
HSA_LOCAL_MEMORY_ENABLE Enable HSA device local memory usage.	1	0: Disable 1: Enable
PAL_ALWAYS_RESIDENT Force memory resources to become resident at allocation time.	0	0: Disable 1: Enable
PAL_PREPINNED_MEMORY_SIZE Size of prepinning memory.	64	Unit: kilobyte (KB)
REMOTE_ALLOC Use remote memory for the	0	0: Disable 1: Enable

Environment variable	Default value	Value
global heap allocation.		

Other useful variables

The following table lists environment variables that are useful but relate to different features in HIP.

Environment variable	Default value	Value
HIPRTC_COMPILE_OPTIONS_APPEND Sets compile options needed for <code>hiprtc</code> compilation.	Unset by default.	- -gpu-architecture=gfx906:sramecc+:x -fgpu-rdc
AMD_COMGR_SAVE_TEMPS Controls the deletion of temporary files generated during the compilation of Comgr. These files do not appear in the current working directory, but are instead left in a platform-specific temporary directory.	Unset by default.	0: Temporary files are deleted automatically Non zero integer: Turn off the temporary file deletion.
AMD_COMGR_EMIT_VERBOSE_LOGS Sets logging of Comgr to include additional Comgr-specific informational messages.	Unset by default.	0: Verbose log disabled. Non zero integer: Verbose log enabled.
AMD_COMGR_REDIRECT_LOGS Controls redirect logs of Comgr.	Unset by default.	<code>stdout</code> / <code>-</code> : Redirected to the standard output <code>stderr</code> : Redirected to the error stream.

ROCR-Runtime environment variables

The following table lists the [ROCR-Runtime](#) environment variables:

Environment variable	Default value	Value
ROCR_VISIBLE_DEVICES Specifies a list of device indices or UUIDs to be exposed to the applications.	None	0, GPU-DEADBEEFDEADBEEF
HSA_NO_SCRATCH_RECLAIM Controls whether scratch memory allocations are permanently assigned to queues or can be reclaimed based on usage thresholds.	0	0: Disable. When dispatches need scratch memory that are lower than the threshold, the memory will be permanently assigned to the queue. For dispatches that exceed the threshold, a scratch-use-once mechanism will be used, resulting in the memory to be unassigned after the dispatch. 1: Enable. If a kernel dispatch needs scratch memory, runtime will allocate and permanently assign device memory to the queue handling the dispatch, even if the amount of scratch memory exceeds the default threshold. This memory will not be available to other queues or processes until this process exits.
HSA_SCRATCH_SINGLE_LIMIT Specifies the threshold for the amount of scratch memory allocated and reclaimed in kernel dispatches.	146800640	0 to 4GB per XCC  latest ▼

Environment variable	Default value	Value
Enabling <code>HSA_NO_SCRATCH_RECLAIM</code> circumvents <code>HSA_SCRATCH_SINGLE_LIMIT</code>, and treats <code>HSA_SCRATCH_SINGLE_LIMIT</code> as the maximum value. NOTE: In the 7.0 release the developer can use the HIP enumerator <code>hipExtLimitScratchCurrent</code> to programmatically change the default scratch memory allocation size. For more information, see Global enums and defines.		
<code>HSA_SCRATCH_SINGLE_LIMIT_ASYNC</code> On GPUs that support asynchronous scratch reclaim, this variable is used instead of <code>HSA_SCRATCH_SINGLE_LIMIT</code> to specify the threshold for scratch memory allocation.	<code>3221225472</code> (3GB)	0 to 4GB per XCC
<code>HSA_ENABLE_SCRATCH_ASYNC_RECLAIM</code> Controls asynchronous scratch memory reclamation on supported GPUs. When enabled, if a device memory allocation fails, ROCr will attempt to reclaim scratch memory assigned to all queues and retry the allocation.	<code>1</code>	0: Disable asynchronous scratch reclaim. 1: Enable asynchronous scratch reclaim on supported GPUs.
<code>HSA_XNACK</code> Enables XNACK.	None	1: Enable
<code>HSA_CU_MASK</code> Sets the mask on a lower level of queue creation in the driver. This mask is also applied to the queues	None	<code>1:0-8</code>  latest ▼

Environment variable	Default value	Value
being profiled.		
HSA_ENABLE_SDMA Enables the use of direct memory access (DMA) engines in all copy directions (Host-to-Device, Device-to-Host, Device-to-Device), when using any of the following APIs: hsa_memory_copy, hsa_amd_memory_fill, hsa_amd_memory_async_copy, hsa_amd_memory_async_copy_on_engine.	1	0: Disable 1: Enable
HSA_ENABLE_PEER_SDMA Note: This environment variable is ignored if <code>HSA_ENABLE_SDMA</code> is set to 0. Enables the use of DMA engines for Device-to-Device copies, when using any of the following APIs: hsa_memory_copy, hsa_amd_memory_async_copy, hsa_amd_memory_async_copy_on_engine.	1	0: Disable 1: Enable
HSA_ENABLE_MWAITX When mwaitx is enabled, on AMD CPUs, runtime will hint to the CPU to go into lower power-states when doing busy loops by using the mwaitx instruction.	1	0: Disable 1: Enable
HSA_OVERRIDE_CPU_AFFINITY_DEBUG Controls whether ROCm helper threads inherit the parent process's CPU affinity mask.	1	0: Enable inheritance. Helper threads use the parent process's core affinity mask, w  latest ▼ with enough cores for all threads.

Environment variable	Default value	Value
		1: Disable inheritance. Helper threads spawn on all available cores, ignoring the parent's affinity settings, which may affect performance in certain environments.
<code>HSA_ENABLE_DEBUG</code> Enables additional debug information and validation in the runtime.	0	0: Disable debug mode. 1: Enable debug mode with additional validation and logging.
<code>HSA_HOTSWAP_DISABLE</code> Stops the runtime from loading the HotSwap tool, which retargets code objects for <code>gfx1250</code> A0 agents. The tool loads by default whenever such an agent is present.	0	Unset, empty, 0, <code>false</code> , <code>off</code> , <code>no</code> , <code>n</code> , or <code>f</code> : Load the HotSwap tool when a supported agent is present. Any other value: Never load the HotSwap tool.
<code>HSA_DISABLE_GFX12_STRICT</code> Controls reporting of the “strict” ISA variant on A0 silicon. Disabled by default, so the agent keeps the base target. Set to 0 to opt in and have the agent report the strict variant instead.	1	0: Report the strict ISA variant on A0 agents. 1, unset, empty, or any other value: Keep the base target on A0 agents.
<code>HSA_HOTSWAP_VERBOSE</code> Enables HotSwap diagnostic logging to stderr. Read by the HotSwap tool itself, not by the runtime, so it has no effect unless the tool is loaded. Errors are always reported regardless of this setting.	0	Unset, empty, or 0: Disable HotSwap diagnostic logging. Any other value: Enable HotSwap diagnostic logging.

Environment variable	Default value	Value
<code>HSA_HOTSWAP_DUMP_SOURCE</code> Writes the source code object to disk when the HotSwap tool refuses to translate it. Off by default because these objects are large and a failed translation is not memoized, so the same bytes fail again on every load. At most one artifact is written per source, for at most 32 distinct sources per process; an out-of-resources failure is never captured.	<code>0</code>	Unset, empty, or 0: Do not write refused code objects. Any other value: Write each refused code object once.
<code>HSA_HOTSWAP_DUMP_DIR</code> Directory that receives the artifacts written by <code>HSA_HOTSWAP_DUMP_SOURCE</code> . Naming a destination does not by itself enable capture.	<code>TMPDIR</code> , else <code>/tmp</code>	Any non-empty path: Write artifacts there. Unset or empty: Fall back to <code>TMPDIR</code> , then <code>/tmp</code> .
<code>HSA_ENABLE_DXG_DETECTION</code> Controls detection of the DXG driver (<code>/dev/dxg</code>) on WSL2.	<code>1</code>	0: Disable DXG detection. 1: Enable DXG detection, allowing ROCr to detect that it is running in WSL2.

Hardware Debugging Environment Variables

The following environment variables are intended for experienced users who are debugging hardware-specific issues. These settings may impact performance and stability and should only be used when troubleshooting specific hardware problems.

Environment variable	Default value	Value
HSA_DISABLE_FRAGMENT_ALLOCATOR Disables internal memory fragment caching to help debug memory faults.	0	0: Fragment allocator enabled (normal operation). 1: Fragment allocator disabled. Helps debug tools identify memory faults at their origin by preventing cached memory blocks from masking out-of-bounds writes.
HSAKMT_DEBUG_LEVEL Controls the verbosity level of debug messages from the <code>libhsakmt.so</code> driver layer.	3	3: Only error messages (<code>pr_err</code>) are printed. 4: Error and warning messages (<code>pr_err</code> , <code>pr_warn</code>) are printed. 5: Same as level 4 (notice level not implemented). 6: Error, warning, and info messages (<code>pr_err</code> , <code>pr_warn</code> , <code>pr_info</code>) are printed. 7: All debug messages including <code>pr_debug</code> are printed.
HSA_ENABLE_INTERRUPT Controls how completion signals are detected, useful for diagnosing interrupt storm issues.	1	0: Disable hardware interrupts. Uses memory-based polling for completion signals instead of interrupts. 1: Enable hardware interrupts (normal operation).
HSA_SVM_GUARD_PAGES Controls the use of guard pages in Shared Virtual Memory (SVM) allocations.	1	0: Disable SVM guard pages (for debugging memory access patterns). 1: Enable SVM guard pages (normal operation).
HSA_DISABLE_CACHE Controls GPU L2 cache utilization for all memory regions.	0	0: Normal caching enabled). 1: Disables L2 cache entirely. Sets all  latest 

Environment variable	Default value	Value
		memory regions as uncacheable (MTYPE=UC) in the GPU, bypassing the L2 cache. Useful for diagnosing cache-related performance or correctness issues.

HIPCC environment variables

This topic provides descriptions of the HIPCC environment variables.

Environment variable	Value
HIP_PLATFORM The platform targeted by HIP. If HIP_PLATFORM isn't set, then HIPCC attempts to auto-detect the platform based on whether the nvcc tool is found.	amd , nvidia
HIP_PATH The path of the HIP SDK on Microsoft Windows for AMD platforms.	Default: C:/hip
ROCM_PATH The path of the installed ROCm software stack on Linux for AMD platforms.	Default: /opt/rocm
CUDA_PATH Path to the CUDA SDK, which is only used for NVIDIA platforms.	Default: /usr/local/cuda
HIP_CLANG_PATH Path to the clang, which is only used for AMD platforms.	Default: ROCM_PATH/llvm/bin or HIP_PATH/../llvm/bin
HIP_LIB_PATH The HIP device library installation path.	Default: HIP_PATH/lib
HIP_DEVICE_LIB_PATH The HIP device library installation path.	
HIPCC_COMPILE_FLAGS_APPEND Append extra flags as compilation options to hipcc .	
HIPCC_LINK_FLAGS_APPEND Append extra flags as compilation options to hipcc .	

Environment variable	Value
<code>HIPCC_VERBOSE</code> Outputs detailed information on subcommands executed during compilation. Note: The <code>--hipcc-verbose=<n></code> command-line option provides equivalent functionality and takes precedence over this variable when both are set.	1: Displays the command to <code>clang++</code> or <code>nvcc</code> with all options (<code>hipcc-cmd</code>). 2: Displays all relevant environment variables and their values. 4: Displays only the arguments passed to the <code>hipcc</code> command (<code>hipcc-args</code>). 5: Displays both the command to <code>clang++</code> or <code>nvcc</code> and <code>hipcc</code> arguments (<code>hipcc-cmd</code> and <code>hipcc-args</code>). 6: Displays all relevant environment variables and their values, along with the arguments to the <code>hipcc</code> command. 7: Displays all of the above: <code>hipcc-cmd</code>, <code>hipcc-args</code>, and environment variables.

Environment variables in ROCm libraries

Many ROCm libraries define environment variables for specific tuning, debugging, or behavioral control. The table below provides an overview and links to further documentation.

Library	Purpose of Environment Variables
hipBLASLt	Manage logging, debugging, offline tuning, and stream-K configuration for hipBLASLt.
hipFile	Control compatibility modes, supported file systems, and statistics collection.
hipSPARSELt	Control logging, debugging and performance monitoring of hipSPARSELt.
rocBLAS	Performance tuning, kernel selection, logging, and debugging for BLAS operations.
rocSHMEM	Control the behavior of rocSHMEM.
rocSOLVER	Control logging of rocSOLVER.
rocSPARSE	Control logging of rocSPARSE.
MIGraphX	Control debugging, testing, and model performance tuning options for MIGraphX.
MIOpen	Control MIOpen logging and debugging, find mode and algorithm behavior and others.
MIVisionX	Control core OpenVX, GPU/device and debugging/profiling, stitching and chroma key configurations, file I/O operations, model deployment, and neural network parameters of MIVisionX.
RCCL	Control the logging, debugging, compiler and assembly behavior, and cache of RPP.
RPP	Logging, debugging, compiler and assembly management, and cache control in RPP
Tensile	Enable testing, debugging, and experime Tensile clients and applications  latest ▼

Key single-variable details

This section provides detailed descriptions, in the standard format, for ROCm libraries that feature a single, key environment variable (or a very minimal set) which is documented directly on this page for convenience.

rocALUTION

Environment variable	Value
ROCALUTION_LAYER If set to 1 , enable file logging. Logs each rocALUTION function call including object constructor/destructor, address of the object, memory allocation, data transfers, all function calls for matrices, vectors, solvers, and preconditioners. The log file is placed in the working directory.	1 (Enable trace file logging) Default: Not set.

[Terms and Conditions](#)

[ROCm Licenses and Disclaimers](#)

[Privacy](#)

[Trademarks](#)

[Supply Chain Transparency](#)

[Fair and Open Competition](#)

[UK Tax Strategy](#)

[Cookie Policy](#)

[Cookie Settings](#)

© 2026 Advanced Micro Devices, Inc